1. 完善程序：

（哥德巴赫猜想）哥德巴赫猜想是指，任一大于 2 的偶数都可写成两个质数之和。迄今为止，这仍然是一个著名的世界难题，被誉为数学王冠上的明珠。试编写程序，验证任一大于 2 且不超过 n 的偶数都能写成两个质数之和。

```
#include <iostream>
using namespace std;

int main()
{
    const int SIZE = 1000;

    int n, r, p[SIZE], i, j, k, ans;
    bool tmp;

    cin>>n;
    r = 1;
    p[1] = 2;
    for (i = 3; i <= n; i++) {
        [    ①    ];
        for (j = 1; j <= r; j++)
            if (i % [     ②    ]  == 0) {
                tmp = false;
                break;
            }
        if (tmp) {
            r++;
            [    ③    ] ;
```

```
        }
    }


    ans = 0;
    for (i = 2; i <= n / 2; i++) {
        tmp = false;
        for (j = 1; j <= r; j++)
            for (k = j; k <= r; k++)
                if (i + i == [     ④   ] ) {
                    tmp = true;
                    break;
                }
        if (tmp)
            ans++;
    }
    cout<<ans<<endl;
    return 0;
}
```

若输入 n 为 2010，则输出[⑤]时表示验证成功，即大于 2 且不超过 2010 的偶数都满足哥德巴赫猜想。

第一空(2.5分): `tmp = true`

第二空(2.5分): `j + 1`

第三空(2.5分): `p[r] = i;`

第四空(2.5分): `p[j] + p[r]`

第五空(3分): `1004`

2.下面是根据欧几里得算法编写的函数，它所计算的是 a 和 b 的(C)。

```
int euclid(int a, int b)
{
   if (b == 0)
      return a;
   else
      return euclid(b, a % b);
}
```

A. 最大公共质因子

B. 最小公共质因子

C. 最大公约数

D. 最小公倍数

3.阅读程序写结果：

```
#include <iostream>
using namespace std;
const int SIZE = 100;
int main()
{
        int p[SIZE];
        int n, tot, i, cn;
        tot = 0;
        cin >> n;
        for ( i = 1; i <= n; i++ )
                p[i] = 1;
        for ( i = 2; i <= n; i++ )
        {
```

```
                if ( p[i] == 1 )
                        tot++;
                cn = i * 2;
                while ( cn <= n )
                {
                        p[cn] = 0;
                        cn += i;
                }
        }
        cout << tot << endl;
        return(0);
}
```

输入：30

输出：__10_

4.10000 以内，与 10000 互质的正整数有（ B ）个。

A. 2000

B. 4000

C. 6000

D. 8000

5.

完善程序

（最大公约数之和）下列程序想要求解整数n的所有约数两两之间最大公约数的和对 10007 求余后的值，试补全程序。举例来说，4 的所有约数是 1, 2, 4 。1 和 2 的最大公约数为 1 ；2 和 4 的最大公约数为 2 ；1 和 4 的最大公约数为 1 。于是答案为$1+2+1=4$。

要求$getDivisor$函数的复杂度为$O(\sqrt{n})$，gcd 函数的复杂度为$O(logmax(a,b))$。

```
#include <iostream>
using namespace std;

const int N = 110000, P = 10007;
int n;
int a[N], len;
int ans;

void getDivisor() {
    len = 0;
    for (int i = 1; ① <= n; ++i)
        if (n % i == 0) {
          a[++len] = i;
          if ( ② != i) a[++len] = n / i;
        }
}

int gcd(int a, int b) {
    if (b == 0) {
         ③ ;
    }
    return gcd(b, ④ );
}

int main() {
    cin >> n;
    getDivisor();
```

```
    ans = 0;
    for (int i = 1; i <= len; ++i) {
        for (int j = i + 1; j <= len; ++j) {
          ans = ( ⑤ ) % P;
        }
    }
    cout << ans << endl;
    return 0;
}
```

第一空(2分): `i * i`　　第二空(3分): `n / i`

第三空(3分): `return a;`　　第四空(3分): `a % b`

第五空(3分): `ans + gcd(a[i], a[j])`

6.100 以内最大的素数是(B).

A. 89

B. 97

C. 91

D. 93

7.319 和 377 的最大公约数是(C)。

A. 27

B. 33

C. 29

D. 31

8.

干支纪年法是中国传统的纪年方法，由10个天干和12个地支组合成60个天干地支。由公历年份可以根据以下公式和表格换算出对应的天干地支。

天干 =（公历年份）除以10所得余数

地支 =（公历年份）除以12所得余数

天干	甲	乙	丙	丁	戊	己	庚	辛	壬	癸		
	4	5	6	7	8	9	0	1	2	3		
地支	子	丑	寅	卯	辰	巳	午	未	申	酉	戌	亥
	4	5	6	7	8	9	10	11	0	1	2	3

例如，今年是 2020 年，2020 除以 10 余数为 0，查表为"庚"；2020 除以 12，余数为 4，查表为"子" 所以今年是庚子年。

请问 1949 年的天干地支是( ).

C

A. 己酉

B. 己亥

C. 己丑

D. 己卯

9.

（质因数分解）给出正整数 n，请输出将 n 质因数分解的结果，结果从小到大输出。 例如：输入 $n=120$，程序应该输出 `2 2 2 3 5`，表示：$120=2\times2\times2\times3\times5$。输入保证 $2\leq n\leq10^9$。提示：先从小到大枚举变量 i，然后用 i 不停试除 n 来寻找所有的质因子。试补全程序。

```
#include <cstdio>
using namespace std;
int n, i;
int main() {
  scanf("%d", &n);
  for(i = ①; ② <=n; i ++){
        ③{
          printf("%d ", i);
          n = n / i;
        }
  }
  if(④)
        printf("%d ", ⑤);
  return 0;
}
```

1）①处应填（ ）
2）②处应填（ ）
3）③处应填（ ）
4）④处应填（ ）
5）⑤处应填（ ）
1.
A. 1
B. `n-1`
C. 2
D. 0
2.
A. `n/i`
B. `n/(i*i)`
C. `i*i`
D. `i*i*i`
3.
A. `if(n%i==0)`
B. `if(i*i<=n)`
C. `while(n%i==0)`
D. `while(i*i<=n)`
4.
A. `n>1`
B. `n<=1`
C. `i<n/i`
D. `i+i<=n`
5.
A. 2
B. `n/i`
C. `n`
D. `i`

第一空(3分)： C　　第二空(3分)： C

第三空(3分)： D　　第四空(3分)： A

第五空(3分)： C

10.

```
1       #include <stdio.h>
2
3       #define n 100000
4       #define  N n+1
5
6       int m;
7       int a[N], b[N], c[N], d[N];
8       int f[N], g[N];
```

```
9
10      void init()
11      {
12              f[1] = g[1] = 1;
13              for (int i = 2; i <= n; i++) {
14                      if (!a[i]) {
15                              b[m++] = i;
16                              c[i] = 1, f[i] = 2;
17                              d[i] = 1, g[i] = i + 1;
18                      }
19                      for (int j = 0; j < m && b[j] * i <= n; j++) {
20                              int k = b[j];
21                              a[i * k] = 1;
22                              if (i % k == 0) {
23                                      c[i * k] = c[i] + 1;
24                                      f[i * k] = f[i] / c[i * k] * (c[i *
k] + 1);
25                                      d[i * k] = d[i];
26                                      g[i * k] = g[i] * k + d[i];
27                                      break;
28                              }
29                              else {
30                                      c[i * k] = 1;
31                                      f[i * k] = 2 * f[i];
32                                      d[i * k] = g[i];
33                                      g[i * k] = g[i] * (k + 1);
34                              }
35                      }
```

```
36              }
37      }
38
39      int main()
40      {
41              init();
42
43              int x;
44              scanf("%d", &x);
45              printf("%d %d\n", f[x], g[x]);
46              return 0;
47      }
```

假设输入的x是不超过1000的自然数，完成下面的判断题和单选题：

•判断题

1）若输入不为"1"，把第12行删去不会影响输出的结果。（ ）

2）第24行`f[i * k] = f[i] / c[i * k] * (c[i * k] + 1)`中的"`f[i]/c[i*k]`"可能存在无法整除而向下取整的情况。（ ）

3）在执行完$init()$后，f数组不是单调递增的，但g数组是单调递增的。（ ）

•单选题

4）$init$函数的时间复杂度为（ ）。

5）在执行完$init()$ 后，$f[1], f[2], f[3].....f[100]$中有（ ）个等于2。

6）当输入"1000"时，输出为（ ）。

1.

A.正确

B.错误

2.

A.正确

B.错误

3.

A.正确

B.错误

4.

A.$O(n)$

B.$O(nlogn)$

C.$O(n\sqrt{n})$

D.$O(n^2)$

5.

A.23

B.24

C.25

D.26

6.

A."15 1340"

B."15 2340"

C."16 2340"

D."16 1340"

第一空(1.5分):	A	第二空(2分):	A
第三空(2分):	B	第四空(3分):	A
第五空(3分):	C	第六空(4分):	C

11.（枚举因数）从小到大打印正整数 n 的所有正因数。

试补全枚举程序。

```
01 #include <bits/stdc++.h>

02 using namespace std;

03
```

```
04 int main() {
05   int n;
06   cin >> n;
07
08   vector<int> fac;
09   fac.reserve((int)ceil(sqrt(n)));
10
11   int i;
12   for (i = 1; i * i < n; ++i) {
13     if (①) {
14       fac.push_back(i);
15     }
16   }
17
18   for (int k = 0; k < fac.size(); ++k) {
19     cout << ② << " ";
20   }
21   if (③) {
22     cout << ④ << " ";
23   }
24   for (int k = fac.size() - 1; k >= 0; --k) {
25     cout << ⑤ << " ";
26   }
27 }
```

1. ①处应填（ ）
   A. n % i == 0
   B. n % i == 1
   C. n % (i-1) == 0
   D. n % (i-1) == 1
2. ②处应填（ ）
   A. n / fac[k]
   B. fac[k]
   C. fac[k]-1
   D. n / (fac[k]-1)
3. ③处应填（ ）
   A. (i-1) * (i-1) == n
   B. (i-1) * i == n
   C. i * i == n
   D. i * (i-1) == n
4. ④处应填（ ）
   A. n-i
   B. n-i+1
   C. i-1
   D. i
5. ⑤处应填（ ）
   A. n / fac[k]
   B. fac[k]
   C. fac[k]-1
   D. n / (fac[k]-1)

答题须知

1.判题时**不**忽略大小写，如答案为B，而你输入的答案为b，则会被判错！

2.判题时**不**会忽略你输入答案的前后空格，如答案为B，而你输入的答案为空格B空格，则会被判错！

3.判断题 A代表正确、B代表错误

第一空(3分)： A　　第二空(3分)： B

第三空(3分)： C　　第四空(3分)： D

第五空(3分)： A